

Introduction: From reactive to thoughtful agents



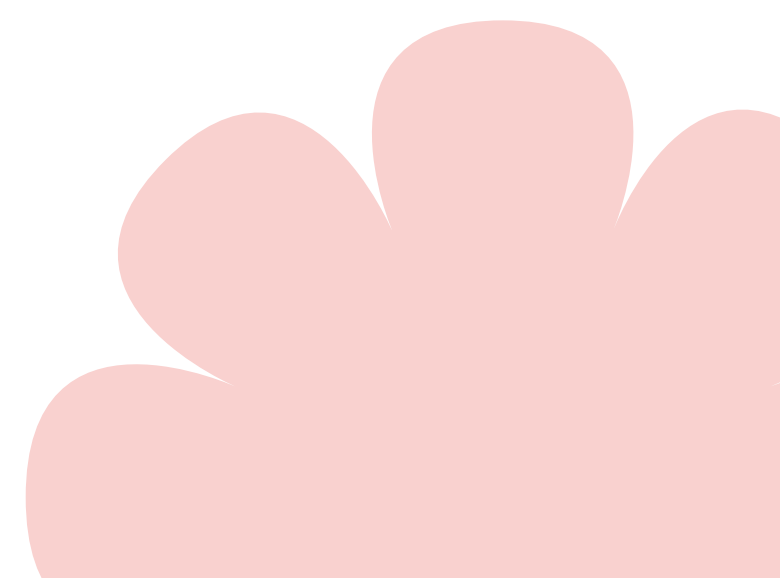
In previous modules, your agents responded immediately:

```
Python
# From Modules 1-3
agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Solve problems for users"
)

# User: "Plan a trip to Japan for 2 weeks with a $5000 budget"
# Agent immediately generates response - might miss considerations
```

This works for simple tasks, but complex problems benefit from structured thinking:

- ✗ No step-by-step problem decomposition
- ✗ No consideration of alternatives
- ✗ No internal reasoning visible to debug
- ✗ Might jump to conclusions without planning



What qualifies as a “complex problem”?

Complex problems require multiple considerations, trade-off analysis, or sequential reasoning:

- **Business strategy**
“How can I reduce cloud costs by 30% without impacting performance?” (requires analyzing cost drivers, performance requirements, and creating a phased approach)
- **Technical decisions**
“Should I use microservices or monolithic architecture for my startup MVP?” (requires weighing trade-offs: speed vs. scalability, team size, future growth)
- **Multi-step planning**
“Plan a 2-week Japan trip for a family of 4 on a \$5000 budget” (requires coordinating flights, hotels, activities, meals—all within budget constraints)

Simple problems that don’t need planning:

- **Direct factual questions**
“What is the capital of France?”
- **Single calculations**
“Convert 100 USD to EUR”
- **Straightforward tasks**
“Greet the user warmly”

Planning vs. multi-agent approaches:

Use planning when a single agent needs to reason through multiple steps or trade-offs within their domain. Use multiple agents (covered in course 5) when the problem requires distinct specialized skills that should be divided among separate agents. This module focuses on adding planning to improve a single agent’s reasoning capabilities.

In this module, we’ll add ADK's [BuiltInPlanner](#) to enable multi-step reasoning.

The problem:

Complex tasks need structured thinking

Without planning, agents struggle with multi-step problems:

Python

```
# No planning - agent tries to solve immediately
problem_solver = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Help users solve complex problems"
)

# User: "How can I reduce my company's cloud costs by 30% without impacting
performance?"
# Agent might:
# - Miss important considerations
# - Provide surface-level suggestions
# - Not think through trade-offs
# - Jump to conclusions
```

The root problem

Complex tasks require breaking down the problem, considering options, and planning an approach before responding.